

17.1: Text Data Files

Text data files, it must be admitted, are not always as compact or as efficient to read and write as binary files. It can be a bit more work to set up the code which reads and writes them. But they have some powerful advantages: any time you need to, you can look at them using ordinary text editors and other tools. If program A is writing a data file which program B is supposed to be able to read but cannot, you can immediately look at the file to see if it's in the correct format and so determine whether it's program A's or B's fault. If program A has not been written yet, you can easily create a data file by hand to test program B with. Text files are automatically portable between machines, even those where integers and other data types are of different sizes or are laid out differently in memory. Because they're not expected to have the rigid formats of binary files, it tends to be more natural to arrange text files so that as the data file format changes slightly, newer (or older) versions of the software can read older (or newer) versions of the data file. Text data files are the focus of this chapter; they're what I use all the time, and they're what I recommend you use unless you have compelling reasons not to.

When we're using text data files, we acknowledge that the *internal* and *external* representations of our data are quite different. For example, a value of type `int` will usually be represented internally as a 2- or 4-byte (16- or 32-bit) piece of memory. Externally, though, that integer will be represented as a string of characters representing its decimal or hexadecimal value. Converting back and forth between the internal and external representations is easy enough. To go from the internal representation to the external, we'll almost always use `printf` or `fprintf`; for example, to convert an `int` we might use `%d` or `%x` format. To convert from the external representation back to the internal, we could use `scanf` or `fscanf`, or read the characters in some other way and then use functions like `atoi`, `strtol`, or `sscanf`.

We have a great many options when it comes to performing this mapping, that is, when converting between the internal and external representations. Our choice may be determined by the layout we want the data file to have, or by what's easiest to implement, or by some combination of these factors. Some of the choices are pretty arbitrary; but in any case, what matters most is obviously that the reading and writing code ``match'', that is, that the data file writing code write the data in the right format such that the data file reading code can accurately read it. For the rest of this section, we'll explore several ways of writing and reading data to and from text data files, using various combinations of the `stdio` functions (and perhaps one or two of our own).

Suppose we had an array of integers:

```
int a[10];
```

and suppose it had been filled up with values, and suppose we wanted to write them out to a data file. We could write them all on one line, separated by spaces:

```
fprintf(ofp, "%d %d %d %d %d %d %d %d %d %d\n",
        a[0], a[1], a[2], a[3], a[4], a[5],
        a[6], a[7], a[8], a[9]);
```

We could write them on 10 separate lines:

```
for(i = 0; i < 10; i++)
    fprintf(ofp, "%d\n", a[i]);
```

Realizing that the loop is easier and more flexible, we could go back to writing them all on one line, using a loop:

```
for(i = 0; i < 10; i++)
    fprintf(ofp, "%d ", a[i]);
fprintf(ofp, "\n");
```

If we were worried about that trailing space at the end of the line, we could arrange to eliminate it:

```
for(i = 0; i < 10; i++)
{
    if(i > 0)
        fprintf(ofp, " ");
    fprintf(ofp, "%d", a[i]);
}
fprintf(ofp, "\n");
```

Recognizing that `fprintf` is overkill for printing single, fixed characters, we could replace two of the calls with `putc`:

```
for(i = 0; i < 10; i++)
{
    if(i > 0)
        putc(' ', ofp);
    fprintf(ofp, "%d", a[i]);
}
putc('\n', ofp);
```

When it came time to read the numbers in, we would have at least as many choices. We could read the ten values all at once, using `fscanf`:

```
int r = fscanf(ifp, "%d %d %d %d %d %d %d %d %d %d",
    &a[0], &a[1], &a[2], &a[3], &a[4], &a[5],
    &a[6], &a[7], &a[8], &a[9]);
if(r != 10)
    fprintf(stderr, "error in data file\n");
```

Since the `scanf` family treats all whitespace (spaces, tabs, and newlines) the same, this code would read either the format with all the numbers on one line, or the format with one number per line. Notice that we check `fscanf`'s return value, to make sure that it successfully read in all the numbers we expected it to. Since data files come in from the outside world, it's possible for them to be corrupted, and programs should not blindly read them assuming that they're perfect. A program that crashes when it attempts to read a damaged data file is terribly frustrating; a program that diagnoses the problem is much more polite.

We could also read the data file a line at a time, converting the text to integers via other means. If the integers were stored one per line, we could use code like this:

```
#define MAXLINE 200

char line[MAXLINE];
for(i = 0; i < 10; i++)
{
    if(fgets(line, MAXLINE, ifp) == NULL)
    {
        fprintf(stderr, "error in data file\n");
        break;
    }
    a[i] = atoi(line);
}
```

(We could also use our own `getline` or `fgetline` function instead of `fgets`.) If the integers were

stored all on one line, we could use the `getwords` function from chapter 10 to separate the numbers at the whitespace boundaries:

```
char *av[10];

if(fgets(line, MAXLINE, ifp) == NULL)
    fprintf(stderr, "error in data file\n");
else if(getwords(line, av, 10) != 10)
    fprintf(stderr, "error in data file\n");
else
    {
        for(i = 0; i < 10; i++)
            a[i] = atoi(av[i]);
    }
```

Suppose, now, that there were not always 10 elements in the array `a`; suppose we had a separate integer variable `na` to record how many elements the array `a` currently contains. When writing the data out, we would certainly then use a loop; we might also want to precede the data by the count, in case that will make it easier for the reading program:

```
fprintf(ofp, "%d\n", na);
for(i = 0; i < na; i++)
    fprintf(ofp, "%d\n", a[i]);
```

We could also print all of the numbers on one line:

```
fprintf(ofp, "%d", na);
for(i = 0; i < na; i++)
    fprintf(ofp, " %d ", a[i]);
```

(Notice that the presence of the extra value at the beginning of the line makes the space separator game easier to play.)

Now, when reading the data in, we would simply read the count first, then the data. Using `fscanf`:

```
if(fscanf(ifp, "%d", &na) != 1)
    {
        fprintf(stderr, "error in data file\n");
        return;
    }

if(na > 10)
    {
        fprintf(stderr, "too many items in data file\n");
        return;
    }

for(i = 0; i < na; i++)
    {
        if(fscanf(ifp, "%d", &a[i]) != 1)
            {
                fprintf(stderr, "error in data file\n");
                return;
            }
    }
```

(Here we assume that the code to read the array from the data file is part of a function, and that when we detect an error, we return early from the function. In practice, we would probably return some error code to the caller.)

If we chose to use `fgets` (or `fgetline`), the code might look like this for data on separate lines:

```
if(fgets(line, MAXLINE, ifp) == NULL)
{
    fprintf(stderr, "error in data file\n");
    return;
}
na = atoi(line);
if(na > 10)
{
    fprintf(stderr, "too many items in data file\n");
    return;
}

for(i = 0; i < na; i++)
{
    if(fgets(line, MAXLINE, ifp) == NULL)
    {
        fprintf(stderr, "error in data file\n");
        return;
    }
    a[i] = atoi(line);
}
```

Or, if the data were all on one line, like this:

```
int ac;
char *av[11];

if(fgets(line, MAXLINE, ifp) == NULL)
{
    fprintf(stderr, "error in data file\n");
    return;
}

ac = getwords(line, av, 10);
if(ac < 1)
{
    fprintf(stderr, "error in data file\n");
    return;
}
na = atoi(av[1]);
if(na > 10)
{
    fprintf(stderr, "too many items in data file\n");
    return;
}
if(na != ac - 1)
{
    fprintf(stderr, "error in data file\n");
    return;
}
for(i = 0; i < na; i++)
    a[i] = atoi(av[i+1]);
```

But sometimes, you don't need to save the count (`na`) explicitly; the reading program can deduce the number of items from the number of items in the file. If the file contains *only* the integers in this array, then we can simply read integers until we reach end-of-file. For example, using `fscanf`:

```
na = 0;
while(na < 10 && fscanf(ifp, "%d", &a[na]) == 1)
    na++;
```

(This code is deceptively simple; we haven't carefully dealt with appropriate error messages for a data file with more than 10 values, or a data file with a non-numeric ``value" for which `fscanf` returns 0.)

Again, we could also use `fgets`. If the data is on separate lines:

```
na = 0;
while(na < 10 && fgets(line, MAXLINE, ifp) != NULL)
    a[na++] = atoi(line);
```

If the data is all on one line:

```
if(fgets(line, MAXLINE, ifp) == NULL)
{
    fprintf(stderr, "error in data file\n");
    return;
}
na = getwords(line, av, 10);
if(na > 10)
{
    fprintf(stderr, "too many items in data file\n");
    return;
}
for(i = 0; i < na; i++)
    a[i] = atoi(av[i]);
```

Notice that this last implementation does not require that the file consist of only data for the array `a`. One *line* of the file consists of data for the array `a`, but other lines of the file could contain other data.

We could also scatter `a`'s data on multiple lines, without using an explicit count, and with the ability for the file to contain other data as well, if we marked the end of the array data with an explicit marker in the file, rather than assuming that the array's data continued until end-of-file. For example, we could write the data out like this:

```
for(i = 0; i < na; i++)
    fprintf(ofp, "%d\n", a[i]);
fprintf(ofp, "end\n");
```

and read it like this:

```
na = 0;
while(fgets(line, MAXLINE, ifp) != NULL)
{
    if(strncmp(line, "end", 3) == 0)
        break;
    if(na > 10)
    {
        fprintf(stderr, "too many items in data file\n");
        return;
    }
    a[na++] = atoi(line);
}
```

(There's just one nuisance here in checking for the ``end" marker: `fgets` leaves the `\n` in the line it reads, so a simple `strcmp` against "end" would fail. Here we use `strncmp`, which compares at most `n` characters, and we pass the third argument, `n`, as 3. Other solutions would be to use `strcmp` against the string "end\n", or to strip the `\n` somehow, or to use our old `getline` or `fgetline` functions, since they strip the `\n` for us.)

Now that we've seen many (too many!) options for writing and reading the array, how do you decide which to use? Should you use `fscanf`, or the slightly more *ad hoc* methods involving `fgets`, `getwords`, `atoi`, etc? It's largely a matter of personal preference. In the code fragments we've looked at so far, the ones using `fscanf` have seemed shorter, although in some cases that was because they weren't doing as much error checking as the ones that used `fgets`. In general, the methods using `fgets` will allow somewhat more flexibility, as we saw when checking for the explicit ``end" marker, which would have been difficult or impossible using `scanf` or `fscanf`.

Now let's move to another example, a user-defined data structure. Suppose we have this structure:

```
struct s
{
    int i;
    float f;
    char s[20];
};
```

To write an instance of this structure out, we could simply print its fields on one line:

```
struct s x;
...
fprintf(ofp, "%d %g %s\n", x.i, x.f, x.s);
```

or on several lines:

```
fprintf(ofp, "%d\n", x.i);
fprintf(ofp, "%g\n", x.f);
fprintf(ofp, "%s\n", x.s);
```

or simply

```
fprintf(ofp, "%d\n%g\n%s\n", x.i, x.f, x.s);
```

(We use `%g` format for the `float` field because `%g` tends to print the most accurate representation in the smallest space, e.g. `1.23e6` instead of `1230000` and `1.23e-6` instead of `0.00000123` or `0.000001`.)

To read this structure back in, we could again either use `fscanf`, or `fgets` and some other functions. As before, `fscanf` seems easier:

```
if(fscanf(ifp, "%d %g %s", &x.i, &x.f, &x.s) != 3)
{
    fprintf(stderr, "error in data file\n");
    return;
}
```

Here we have a problem, though: what if the third, string field contains a space? In the `scanf` family, the `%s` format stops reading at whitespace, so if `x.s` had contained the string "Hello, world!", it would be read back in as "Hello, ". As it happens, we could fix it by using the less-obvious format string `"%d %g %[^\\n]"`, where `%[^\\n]` means ``match any string of characters not including `\\n`". But we also have another problem: what if the string is longer than the 20 characters we allocated for the `s` field? We could fix this by using `%20s` or `%20[^\\n]`, although we'd have to remember to change the `scanf` format string if we ever changed the size of the array.

Let's leave `fscanf` for a moment and look at our other alternatives. If we'd printed the data all on one line, we could use

```

#include <stdlib.h>      /* for atof() */

char *av[3];

if(fgets(line, MAXLINE, ifp) == NULL)
{
    fprintf(stderr, "error in data file\n");
    return;
}
if(getwords(line, av, 3) != 3)
{
    fprintf(stderr, "error in data file\n");
    return;
}
x.i = atoi(av[0]);
x.f = atof(av[1]);
strcpy(x.s, av[2]);      /* XXX */

```

Here we luck out on the question of what happens if the string contains a space, because it happens that our version of `getwords` (see chapter 10, p. 13) leaves the remaining words in the last “word” if there are more words in the string than we told it to find, i.e. more than the third argument to `getwords` which gives the size of the `av` array. Here, we told it it could only look for 3 words, so if the string contains spaces, making the line appear to have 4 or more words, words 3, 4, etc. will all be pointed to by `av[2]`. However, we still have the problem that we haven't guarded against overflow of `x.s` if the third (plus fourth, etc.) word on the data line is longer than 20 characters. (The comment `/* XXX */` is a traditional marker which means “this line is inadequate and definitely won't work reliably in all situations but for one reason or another the person writing it is not going to take the trouble to do it right just yet.”)

If the data is written on three lines, on the other hand, we obviously have to call `fgets` three times to read it:

```

if(fgets(line, MAXLINE, ifp) == NULL)
{ fprintf(stderr, "error in data file\n"); return; }
x.i = atoi(line);

if(fgets(line, MAXLINE, ifp) == NULL)
{ fprintf(stderr, "error in data file\n"); return; }
x.f = atof(line);

if(fgets(line, MAXLINE, ifp) == NULL)
{ fprintf(stderr, "error in data file\n"); return; }
strcpy(x.s, line);      /* XXX */

```

Now the last line has two problems: besides the lingering problem of overflow (if the line is more than 18 characters long), we have the problem that `fgets` retains the `\n` (which is why `x.s` will overflow if the line is longer than 18 characters, not 19). In this case, one way to fix the overflow problem would be to have `fgets` read into `x.s` directly:

```

if(fgets(x.s, 20, ifp) == NULL)
{ fprintf(stderr, "error in data file\n"); return; }

```

If we didn't want to have to remember to change that 20 in the call to `fgets` if we ever re-sized the array, we could get clever and write `fgets(x.s, sizeof(x.s), ifp)`. Also, we might as well figure out how to get rid of that pesky `\n`. One way is by calling the standard library function `strchr`, which searches for a certain character in a string. This will require that we `#include <string.h>`, and declare an extra `char *` variable:

```
#include <string.h>
char *p;
p = strchr(x.s, '\n');
if (p != NULL)
    *p = '\0';
```

`strchr` returns a pointer to the character that it finds, or a null pointer if it doesn't find the character. If there's a `\n` in the line at all, we know it's at the end, so it's safe to overwrite it with a `\0`, making the string one character shorter. (Since we know that the `\n` is at the end, we could also call the function `strrchr`, which finds a character starting from the right.)

For any of the methods we've been using so far, what if one day we add a new field to the structure `s`? Obviously, we'll have to rewrite the code which writes the structure out and also the code which reads it in. Also, unless we're careful, the modified code won't be able to read in any data files we might happen to have lying around which were written before the structure was changed. Depending on the nature of the data file and the way it's used, this can be a real problem. (In principle, it's possible to write a utility program to convert the old data files to the new format, but it can be a nuisance to write that program, and it can be a *real* nuisance to track down all of the old data files that need converting.)

Therefore, when a data file format must be changed, it's often a good idea if the new, improved data file reader can be made to automatically detect and read old-format files as well. (Automatic detection isn't a strict necessity, but it's certainly a nicety.) Furthermore, it's *much* easier to write a new & improved data file reader, that can read both old and new formats, if the possibility was thought of back when the original data file format was designed.

One thing that helps a lot is if data file formats have version numbers, and if each data file begins with a number, in a simple format and known location which won't change even if the rest of the format changes, indicating which version of the format this file uses. Having a file format version number at the beginning of each data file leads to two immediate advantages:

1. Whenever a new program reads a data file, it can immediately and unambiguously decide how it's going to read it, whether it can use its new & improved reading routines or whether it might have to fall back on its backwards-compatibility, old-style reader.
2. If there is a suite of several programs, all of which read the same data files, and if for some reason there's an old version of one of the programs still in use, the old program can print an unambiguous message along the lines of ``this is a new data file which I am too old to read'', rather than printing the (misleading, in this case) ``error in data file" (or crashing).

Another technique which can be immensely useful and which we'll explore next is to define a data file format in such a way that the overall format doesn't change even if new data is added to it.

It's easy to see why the simple data file fragments we've been looking at so far are not resilient in the face of newly-introduced data fields. In the case of `struct s`, the reader always assumed that the first field in the data file was `i`, the second field was `f`, and the third field was `s`. If we ever add any new fields, unless we're careful to add them at the end of the file (and lucky on top of that), the simpleminded reader will get confused.

One powerful way of getting around this problem is to *tag* each piece of data in the file, so that the reader knows unambiguously what it is. For example, suppose that we wrote instances of our `struct s` out like this:

```
fprintf(ofp, "i %d\n", x.i);
fprintf(ofp, "f %g\n", x.f);
fprintf(ofp, "s %s\n", x.s);
```


Now, each line begins with a little code which identifies it. (The code in the data file happens to match the name of the corresponding structure member, but that's not necessary, nor is there any way of getting the compiler to make any correspondence automatically.)

If we simply modified one of our previous file-reading code fragments to read this new, tagged format, we might quickly end up with a mess. We'd be continually checking the tag on the line we just read against the tag we expected to read, and constantly printing error messages or trying to resynchronize. But in fact, there's no reason to expect the lines to come in a certain order, and it turns out that it's easier to read such a file a line at a time, without that assumption, taking each line as it comes and not worrying what order the lines come in. Here is how we might do it:

```
x.i = 0; x.f = 0.0; x.s[0] = '\0';

while(fgets(line, MAXLINE, ifp) != NULL)
{
    if(*line == '#')
        continue;
    ac = getwords(line, av, 2);
    if(ac == 0)
        continue;
    if(strcmp(av[0], "i") == 0)
        x.i = atoi(av[1]);
    else if(strcmp(av[0], "f") == 0)
        x.f = atof(av[1]);
    else if(strcmp(av[0], "s") == 0)
        strcpy(x.s, av[1]);    /* XXX */
}
```

This example also throws in a few new little features: a line beginning with # is ignored, so we will be able to place comment lines in data files by beginning them with #. The code also ignores blank lines (those for which `getwords` returns 0).

We're now treating the "data file" almost like a "command file"--the first word on each line is almost like a "command" telling us to do something: `i` means store this value in `x.i`; `f` means store this value in `x.f`, etc. Since we don't have any easy way of telling whether we ever got around to setting a particular field, we initialize each one to an appropriate default value before we start. Notice that we did *not* have a last line in the `if/else/if/else` chain saying

```
else    fprintf(stderr, "error in data file\n");
```

Instead, we quietly *ignore* lines we don't recognize! This strategy is admittedly on the simpleminded side, and it would not be adequate under all circumstances, but it means that an old program can read a new data file containing fields it's never heard of. The old program will still be able to pluck out the data it does recognize and can use, while (deliberately) ignoring the (new) data it doesn't know about.

This code is not perfect. We still have the same sorts of problems with that string field, `s`: it might contain spaces, which we get around (this time) by calling `getwords` with a second argument of 2, so that all but the first word on the line end up "in" `av[1]`. Also, the code does not check to see that there actually was a second word on the line before using it to set `x.i`, `x.f`, or `x.s`. (In this case, we could fix that by complaining if `getwords` did not return 2.)

Finally, we still have the potential for overflow, and we might as well grit our teeth now and figure out how to fix it. Since we already initialized `x.s` to the empty string with the assignment `x.s[0] = '\0'`, one way around the problem is to replace the call to `strcpy` with a call to `strncat`:

```
...
else if(strcmp(av[0], "s") == 0)
    strncat(x.s, av[1], 19);
```

(or, again, perhaps `strncat(x.s, av[1], sizeof(x.s)-1)`). The `strcat` and `strncat` functions are slightly misleadingly named: what they actually do is *append* the second string you hand them (i.e. the second argument) to the first, in place. In the case of `strncat`, it never copies more than `n` characters, where `n` is its third argument, although it does always append a `\0`, which is why we tell it to copy at most 19 characters, not 20. (Since `x1.s` starts out empty, there's definitely room for 19, although we would still have to worry about the possibility of a corrupted data file which contained two `s` lines. You might wonder why we couldn't simply use `strncpy`, but it turns out that, for obscure historical reasons, `strncpy` does *not* always append the `\0`.)

Although it has a few imperfections (which are easily remedied, and are left as exercises) this last example (using `fgets`, `getwords`, and an `if/strcmp/else...` chain) is an excellent basis for a flexible, robust data file reader.

One footnote about the troublesome string field, `s`: to get around the problem of fixed-size arrays, you might one day decide to declare the `s` field of `struct s` as a pointer rather than a fixed-size array. You would have to be careful while reading, however. It might seem that you could just write, for example,

```
x.s = av[1];    /* assumes char *s, but also WRONG */
```

but this would *not* work; remember that whenever you use pointers you have to worry about memory allocation. If you assigned `x.s` in that way, where would be the memory that it points to? It would be wherever `av[1]` points, which is back into the `line` array. Not only is that (probably) a local array, valid only while the file-reading functions are active, but it's also overwritten with each new line in the data file. You'll obviously want `x.s` to retain a useful pointer value pointing to the text read from the file, which means that you'll still have to make a copy, after allocating some memory. In this case, you might do

```
x.s = malloc(strlen(av[1]) + 1);
if(x.s == NULL)
    { fprintf(stderr, "out of memory\n"); return; }
strcpy(x.s, av[1]);
```

To some extent, the problems we've been having with field `s` are fundamental. In particular, any time you use text formats which are based on whitespace-separated ``words," string fields which might *contain* spaces are always tricky to handle.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1996-1999 // [mail feedback](#)